



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

April 19, 2026

Content in this Practice I

1 Heap

- Tree Structures
- Heap
- Insertion
- Deleting the Maximum Element

2 Priority Queue

- Insertion
- Remove Maximum
- Analysis
- Exercise

3 Heap Sort

- Heap Sort
- Exercise
- Analysis

Content in this Practice II

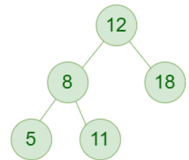
- More Efficient Implementation

Heap

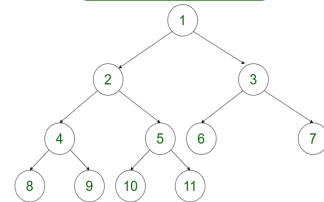
Tree Structures

- **Strictly binary tree:** (Full binary tree)
Every internal node has exactly two children.
- **Nearly complete binary tree:**
A full binary tree where some leaves (but not all) are omitted from the lowest level.
- **Complete binary tree:** (Level-continuous tree)
A nearly complete binary tree where the leaves on the lowest level are filled from left to right without any gaps.
- **Perfect binary tree:**
A strictly binary tree where all internal nodes have two children and all leaves are at the same level.

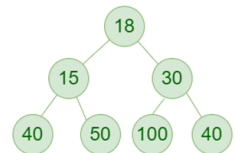
Full Binary Tree



Complete Binary Tree

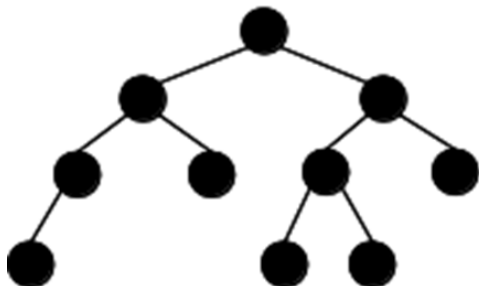


Perfect Binary Tree

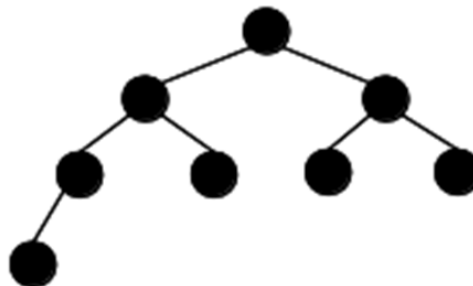


Tree Structures

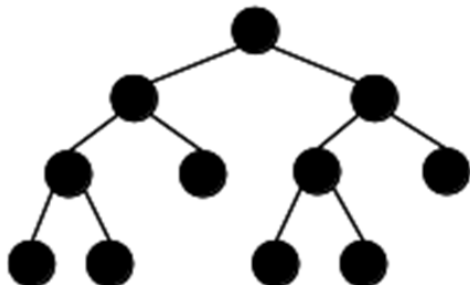
Neither complete nor full



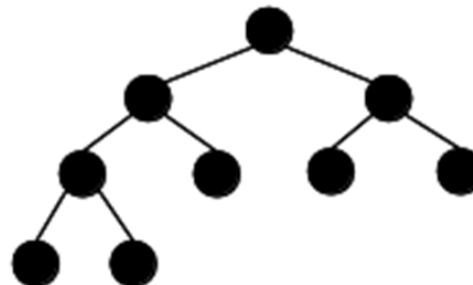
Complete but not full



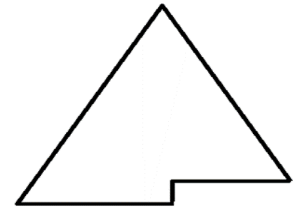
Full but not complete



Complete and full

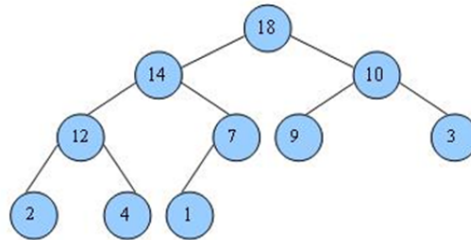


- **Definition of a Heap:** A nearly complete, left-justified (complete) binary tree.
- **Max-heap:** The key of every internal node is greater than or equal to the keys of its children. The largest element is always at the root.
- **Min-heap:** The key of the parent is less than or equal to the keys of its children.



Definition of the Max-Heap

- A binary **max-heap** is a complete binary tree where every node's key is greater than or equal to ($\geq$) the keys of its children.
- Duplicate keys are allowed in the heap.
- The maximum element is directly accessible: it is the root.
- (A **min-heap** is defined analogously ($\leq$).)
- The height of a complete binary tree with n nodes is $\lfloor \log_2 n \rfloor$.

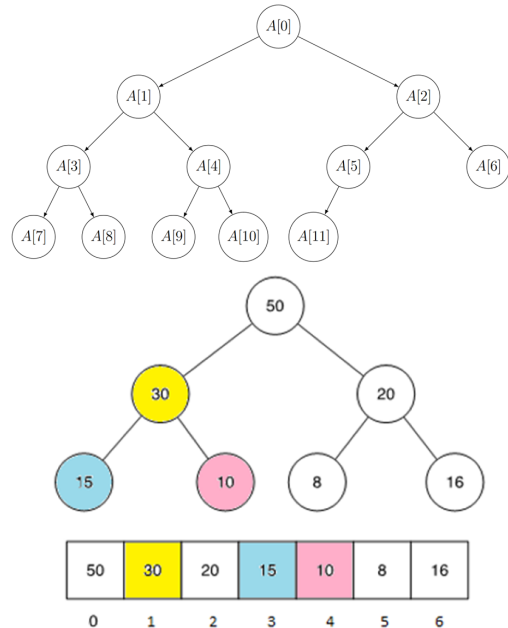


Arithmetic Representation

Heaps (and complete binary trees) can be efficiently represented with arrays, as level-order traversal provides a unique and gapless enumeration.

Array indexing starts from 0 ($A[0..n-1]$):

- Index of a node: i
- Left child's index: $2i + 1$
- Right child's index: $2i + 2$
- Parent's index: $\lfloor \frac{i-1}{2} \rfloor$



Heap Operations

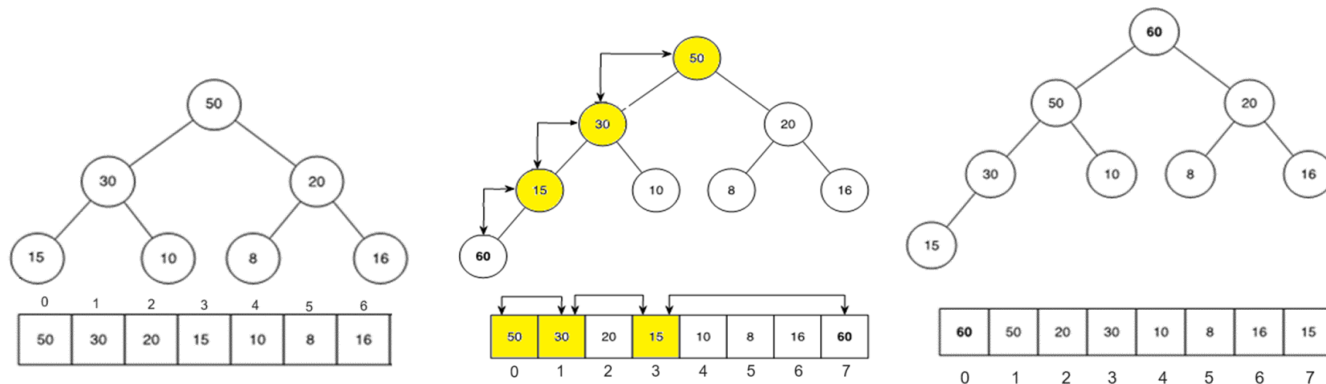
- **Two primary operations:**
 - Insertion (`add`)
 - Remove maximum (`remMax`)
- The heap is represented arithmetically: $A : T[n]$.
- Empty spaces may remain at the end of the array, but there are no "gaps" between elements.

Insertion (add)

Example: Insert 60 into the heap (50, 30, 20, 15, 10, 8, 16)!

- 1 **Step 1:** Place 60 at the very end of the array (at the first free position).
- 2 **Step 2:** The new element "rises" (*rise*) until the heap property is restored (via swaps with its parent).

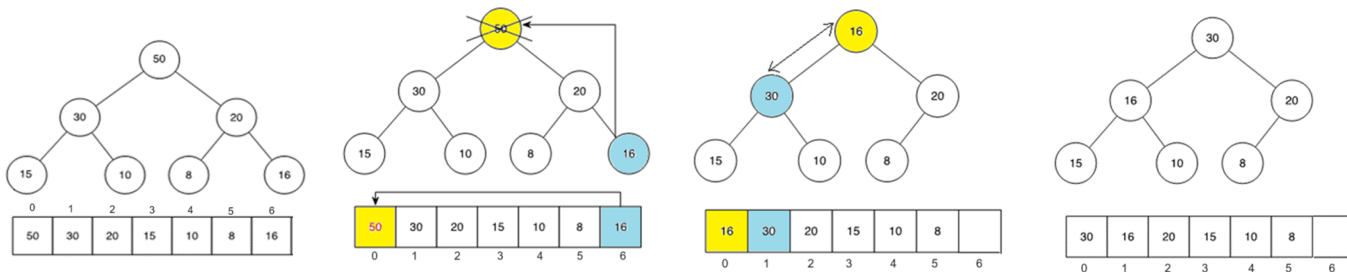
This requires at most $\lfloor \log_2 n \rfloor$ swaps.



Deleting the Maximum Element

- 1 After removing the maximum (root), move the last element of the array into the root. This maintains the continuity of the array.
- 2 Then the element "sinks" (*sink*): as long as it is smaller than either of its children, swap it with the larger child.

Maximum number of swaps: $\lfloor \log_2 n \rfloor$.



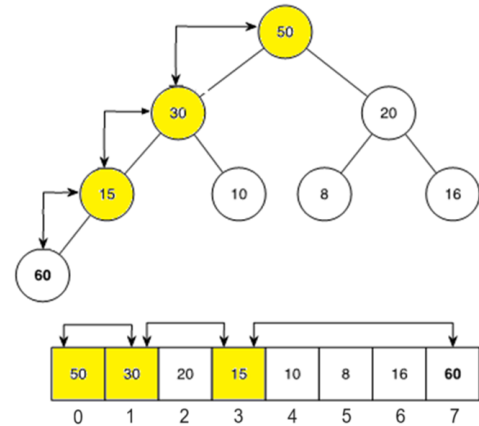
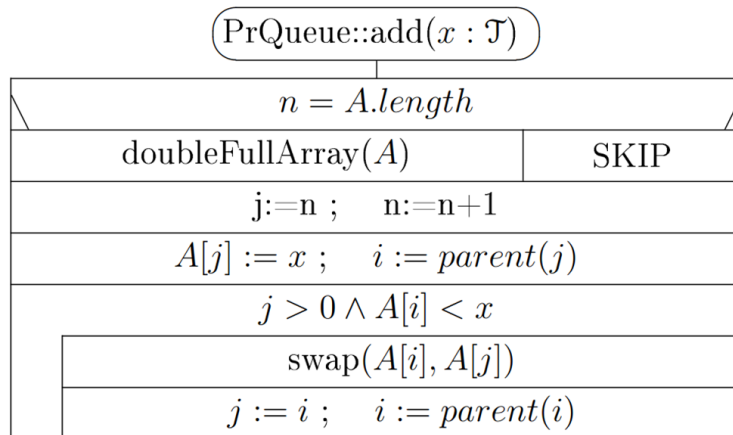
Priority Queue

Priority Queue

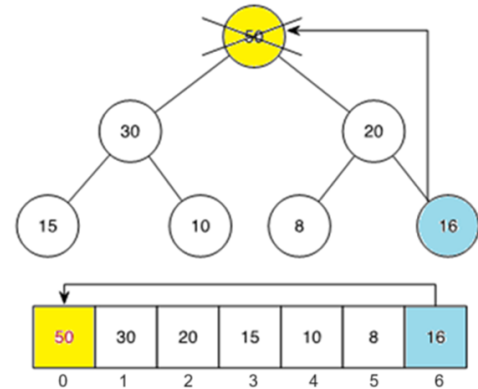
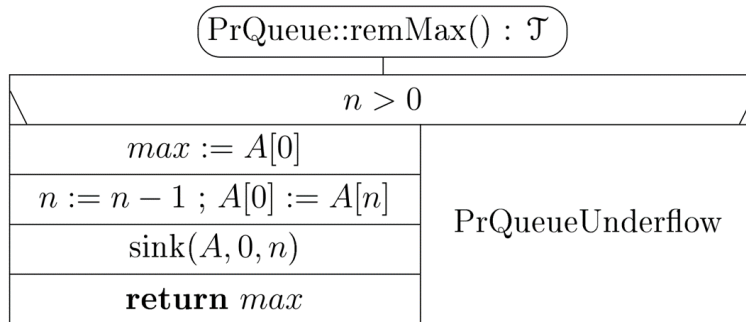
- A queue from which the highest (or lowest) priority element is always removed.
- Can be efficiently implemented using a heap.

PrQueue
- $A/1 : \mathcal{T}[]$ // $\mathcal{T}$ is some known type
- $n : \mathbb{N}$ // $n \in [0..A.length]$ is the actual length of the priority queue
+ PrQueue($m : \mathbb{N}$) { $A := \text{new } \mathcal{T}[m]; n := 0$ } // create an empty priority queue
+ add($x : \mathcal{T}$) // insert x into the priority queue
+ remMax(): $\mathcal{T}$ // remove and return the maximal element of the priority queue
+ max(): $\mathcal{T}$ // return the maximal element of the priority queue
+ isFull() : $\mathbb{B}$ {return $n = A.length$ }
+ isEmpty() : $\mathbb{B}$ {return $n = 0$ }
+ $\sim$ PrQueue() { delete A }
+ setEmpty() { $n := 0$ } // reinitialize the priority queue

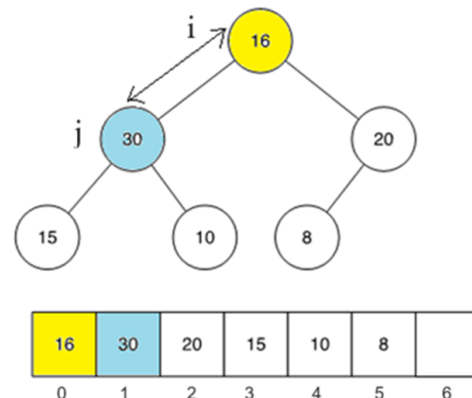
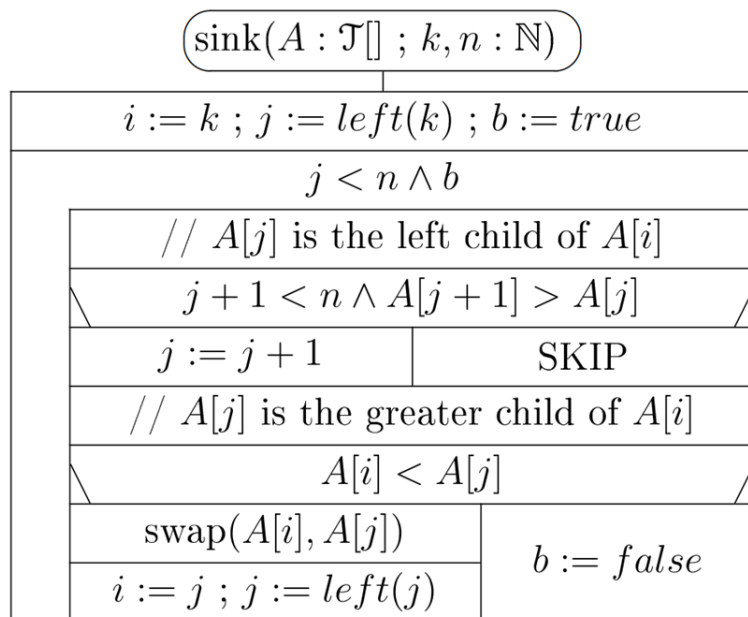
Priority Queue - Insertion



Priority Queue - Remove Maximum



Priority Queue - Sink



- Sink(A,1,6)
- Let the index of the node be: i
- Index of its left child: $2 * i$
- Index of its right child: $2 * i + 1$
- Index of its parent: $\lfloor \frac{i}{2} \rfloor$ ($i/2$ floor value)

Comparison of Max-Priority Queue Implementations

	<code>add(x)</code>	<code>remMax()</code>	<code>max()</code>
Unordered array (tracking max)	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
Ordered array (descending)	$O(n)$	$\Theta(1)$	$\Theta(1)$
Heap	$O(\log n)$	$O(\log n)$	$\Theta(1)$

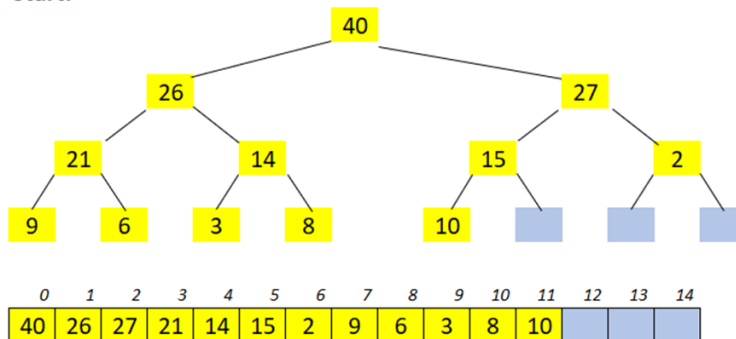
Exercise: Heap

Given the following priority queue:

[40, 26, 27, 21, 14, 15, 2, 9, 6, 3, 8, 10, _, _, _].

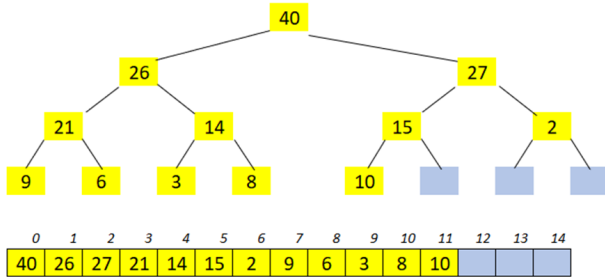
- Demonstrate the `add(61)` and subsequent `add(43)` operations!
- Demonstrate two `remMax()` operations starting from the original heap!
- Illustrate the process in both tree and array form!

Start:

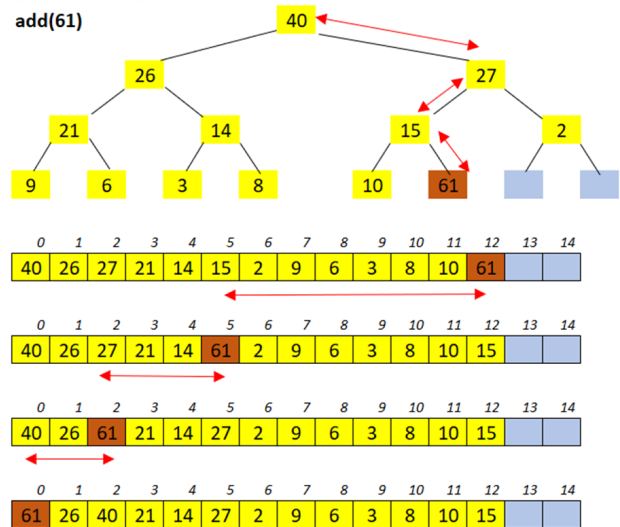


Solution: Heap (Q.add(61))

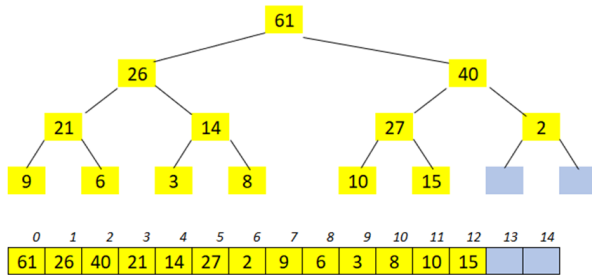
Start:



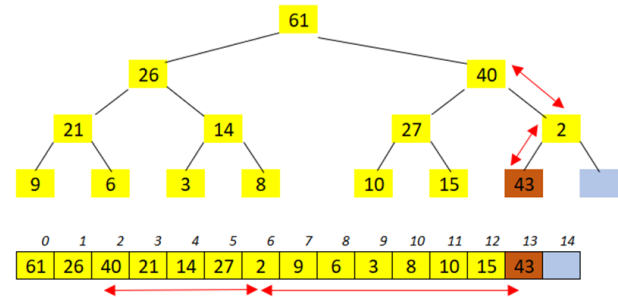
Insertion of 61:
add(61)



Solution: Heap (Q.add(43))

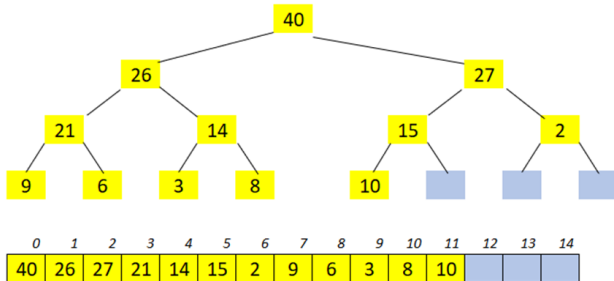


add(43):

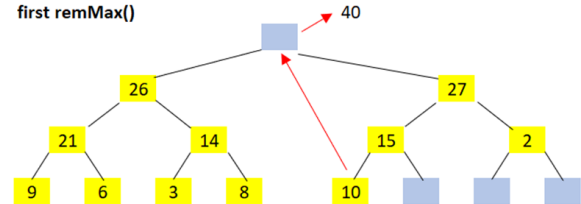


Solution: Removing the Maximum (First Q.remMax():)

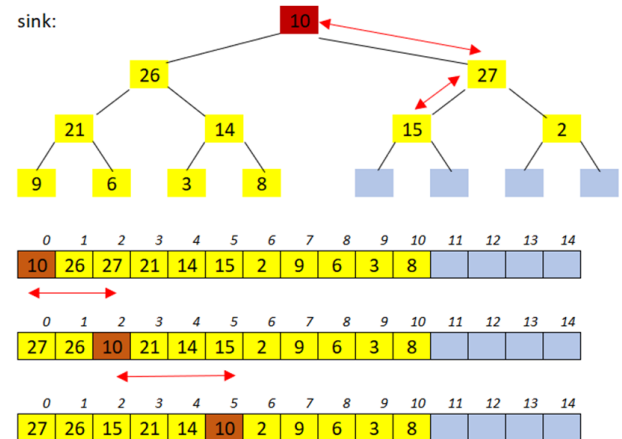
Start



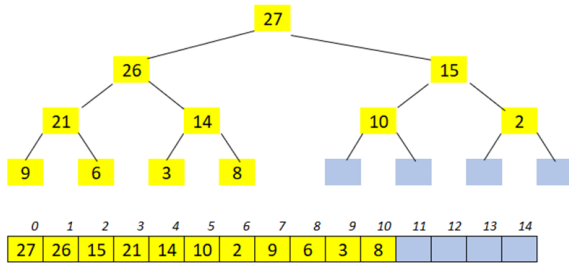
first remMax()



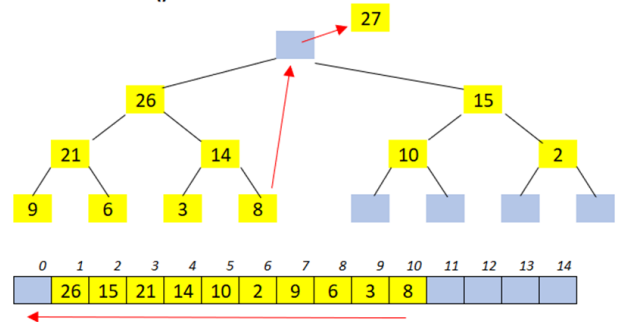
sink:



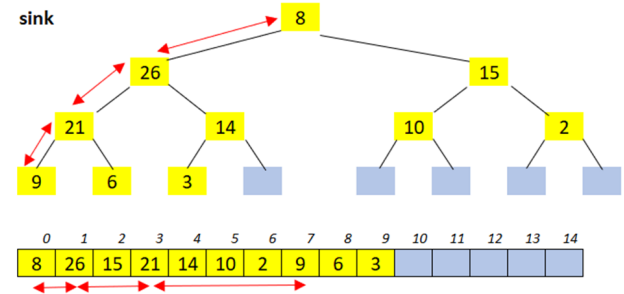
Solution: Removing the Maximum (Second Q.remMax():)



second remMax():



sink



Optional Exercise: Optimization

The efficiency of sinking (*sink*) and rising (*rise*) can be improved:

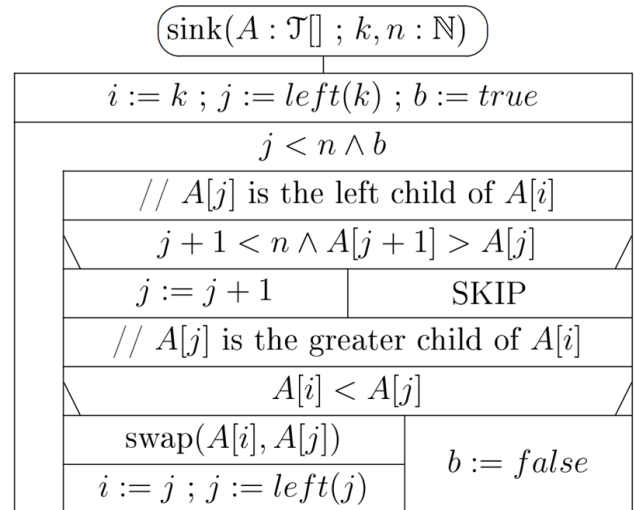
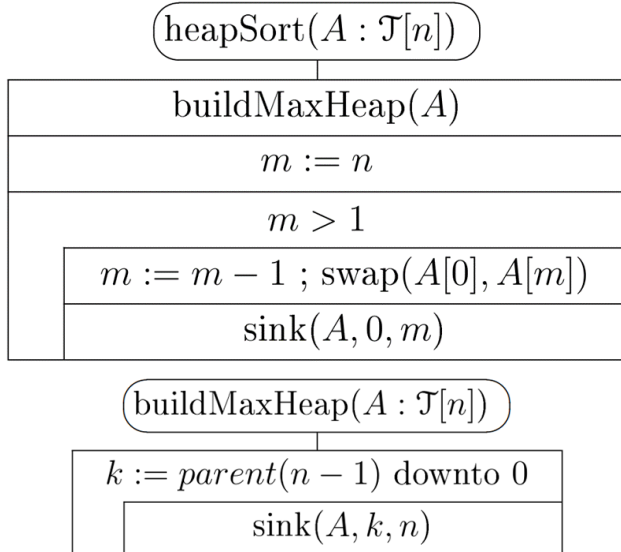
- 1 Implement the **sink** procedure without direct swaps (**swap**)! Use a helper variable for the element being moved and only copy the other elements until the final position is found.
- 2 Implement the **rise** procedure in a similar way.

Heap Sort

Heap Sort

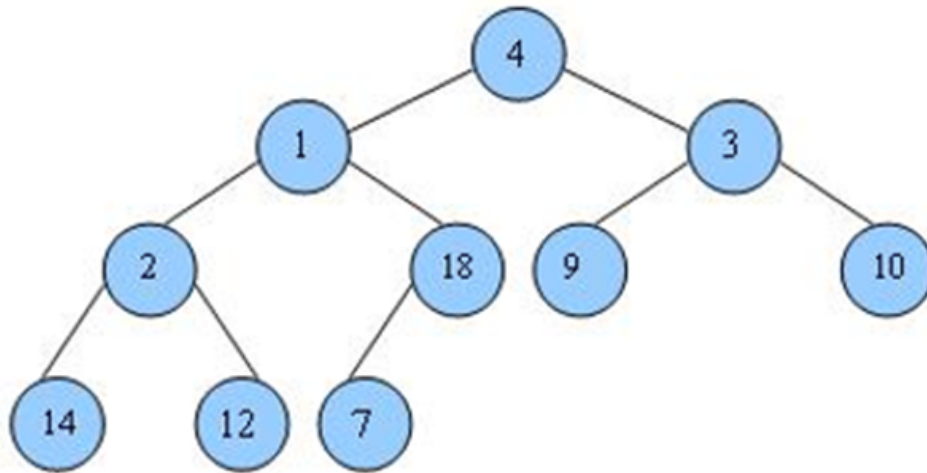
- **Time complexity:** $O(n \log n)$.
- **Space complexity:** In-place sorting, requires no extra storage.
- **Operation:**
 - ① Build a heap from the array (`buildMaxHeap`).
 - ② Move elements to their final positions (via `n` `remMax` sink operations).
- **Building the Heap (`buildMaxHeap`):**
 - Perform $n/2$ sink (`sink`) operations.
 - Indexes proceed from back to front: from $k = \lfloor \frac{n-1}{2} \rfloor$ down to 0.
 - **Complexity:** $\Theta(n)$ (more efficient than performing n insertions, which would be $O(n \log n)$).

Heap Sort



Exercise: Heap Sort

- Demonstrate heap sort on the following array:
- [4, 1, 3, 2, 18, 9, 10, 14, 12, 7].
- The array contains the following level-continuously stored binary tree:



Building the Heap

Step 1.: Building max heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7
k					$n-1$				

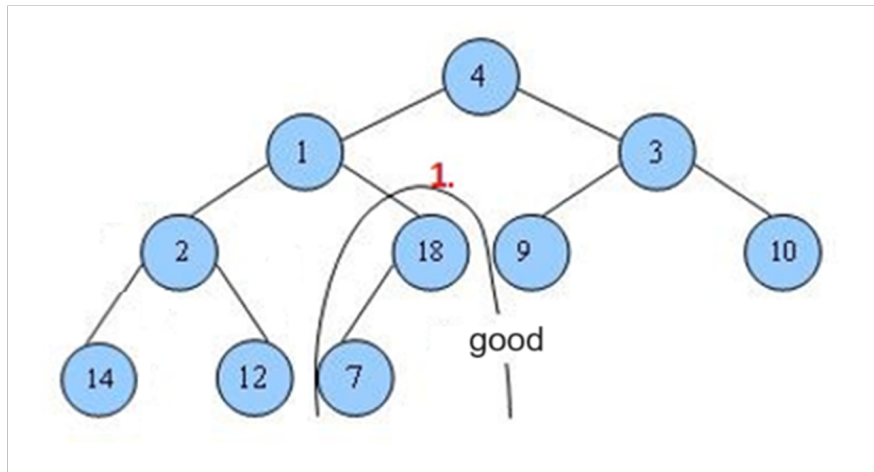
$$k := \left\lfloor \frac{n-1}{2} \right\rfloor = 4$$

1.
 $k=4$
 $\text{Sink}(A, 4, 10)$

$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n-1)$ downto 0

$\text{sink}(A, k, n)$

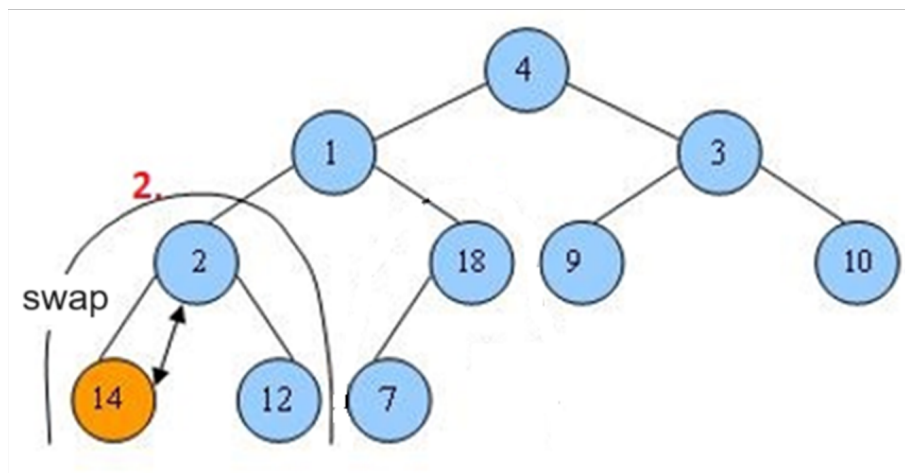
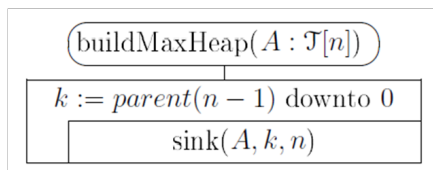


Building the Heap

Building max
heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7

2.
 $k=3$
 $\text{Sink}(A, 3, 10)$



Building the Heap

Building max
heap

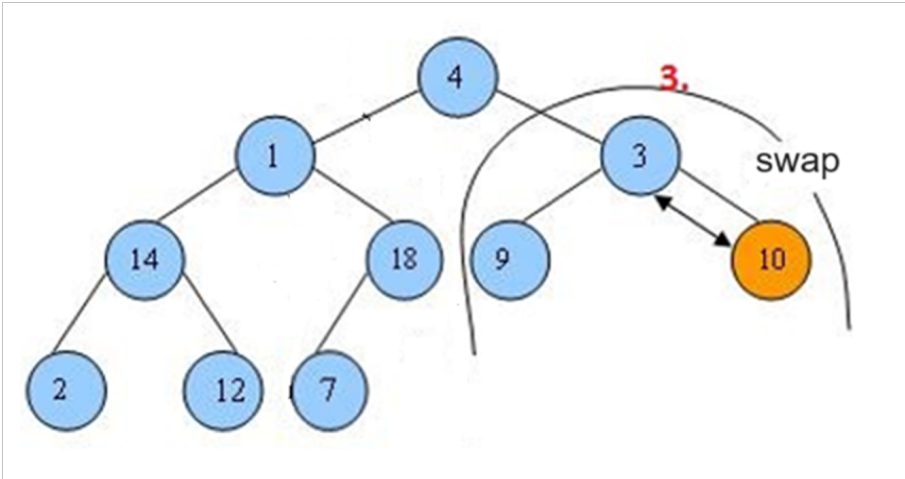
0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7

3.
k=2
Sink(A,2,10)

buildMaxHeap($A : \mathcal{T}[n]$)

$k := \text{parent}(n - 1) \text{ downto } 0$

 sink(A, k, n)



Building the Heap

Building max
heap

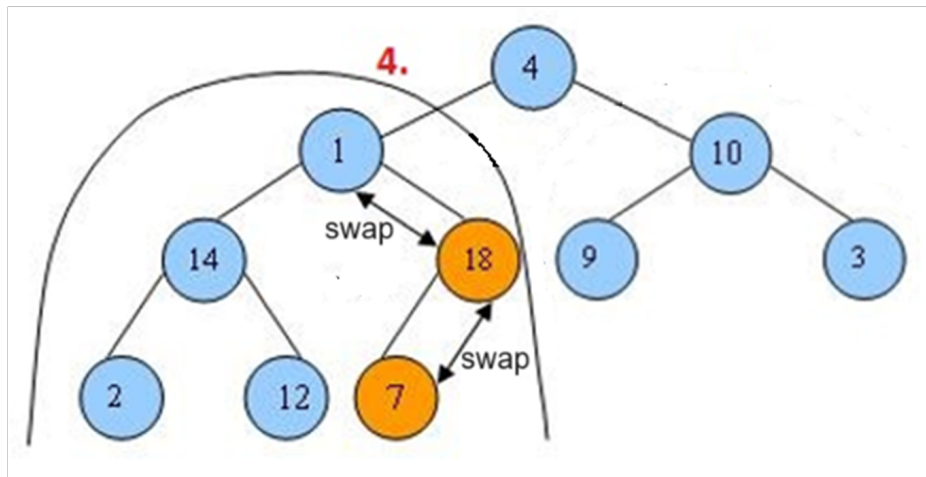
0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7

4.
k=1
Sink(A,1,10)

buildMaxHeap($A : \mathcal{T}[n]$)

$k := \text{parent}(n - 1) \text{ downto } 0$

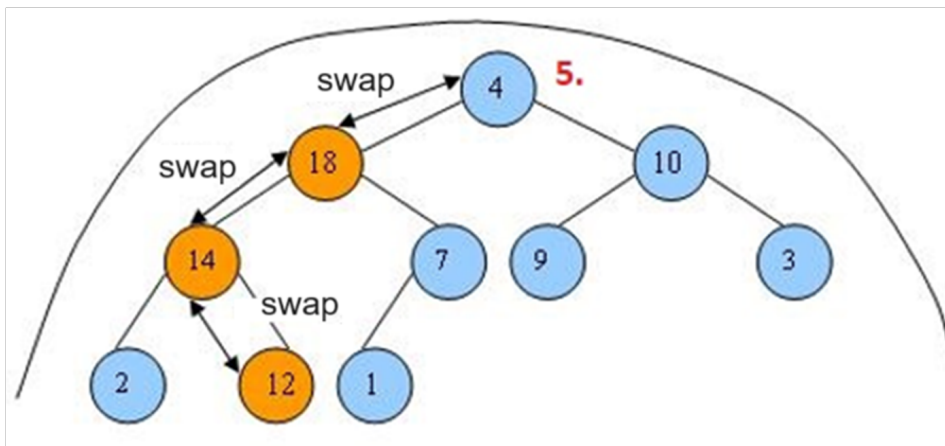
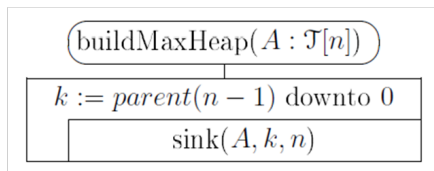
 sink(A, k, n)



Building the Heap

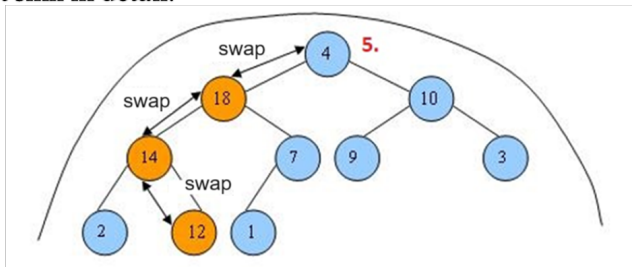
Building max heap

5.
k=0
Sink(A,0,10)

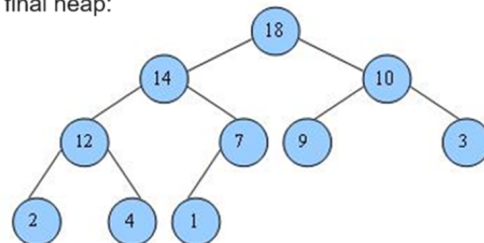


Building the Heap

Last sink in detail:

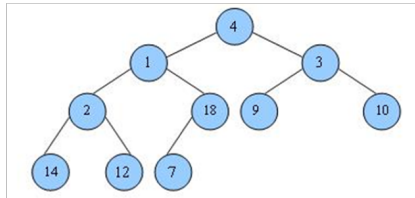


final heap:



0	1	2	3	4	5	6	7	8	9
4	18	10	14	7	9	3	2	12	1
18	4	10	14	7	9	3	2	12	1
18	14	10	4	7	9	3	2	12	1
18	14	10	12	7	9	3	2	4	1

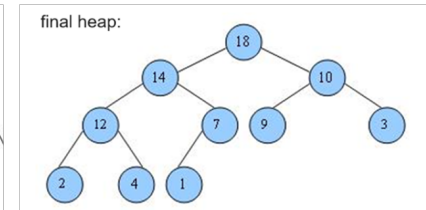
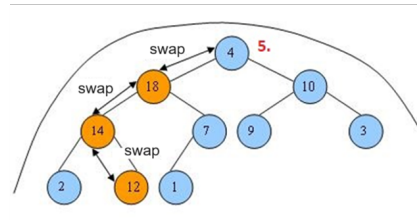
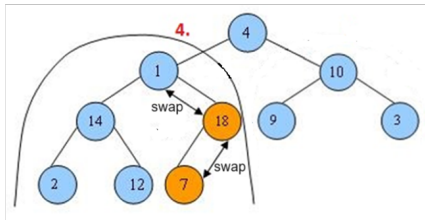
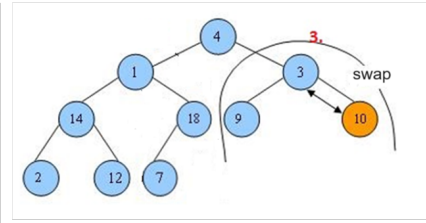
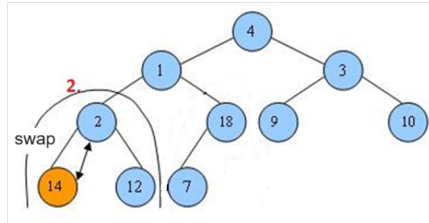
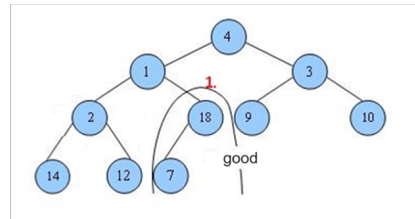
Building the Heap



```

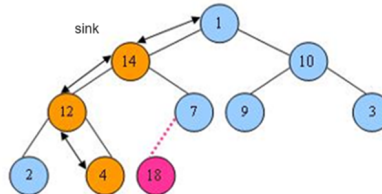
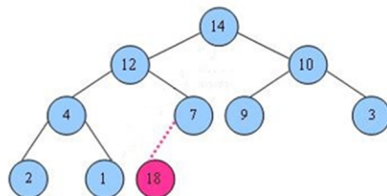
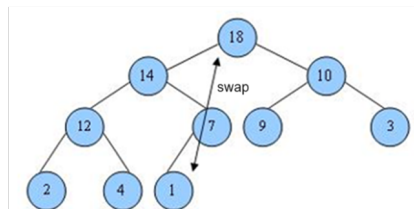
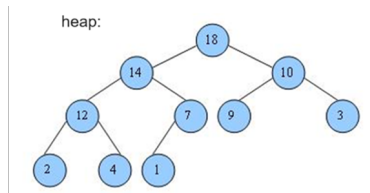
buildMaxHeap( $A : \mathcal{T}[n]$ )
├──  $k := \text{parent}(n - 1) \text{ downto } 0$ 
│   └──  $\text{sink}(A, k, n)$ 

```



Sorting Phase

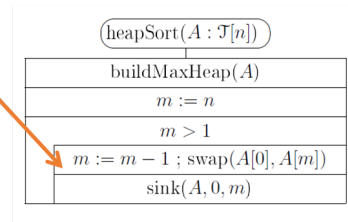
Step 2. Sorting



By the end of the first iteration, the heap structure is good, and the second largest element (14) is at the top. 18 is still in the array, but logically not part of the heap.

0	1	2	3	4	5	6	7	8	9
14	12	10	4	7	9	3	2	1	18

- ◆ We will only present two steps of this sorting part.

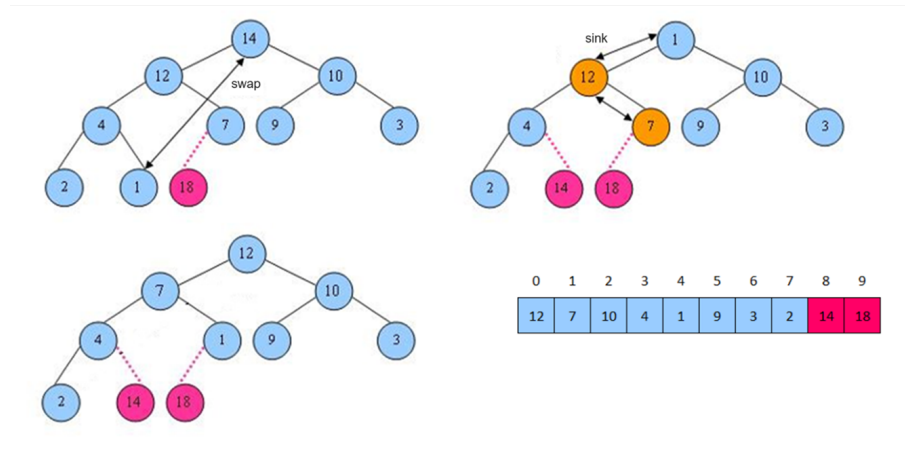


$m=10 \rightarrow m=9$

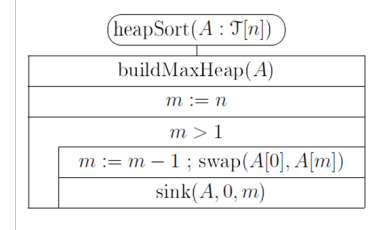
$\text{swap}(A[0], A[9])$

$\text{sink}(A, 0, 9)$

Sorting Phase



We will stop here, but the algorithm goes on until $m=1$.



$m=9 \rightarrow m=8$

$\text{swap}(A[0], A[8])$

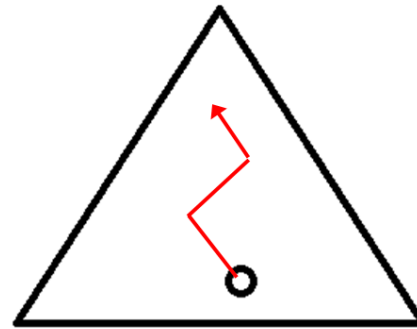
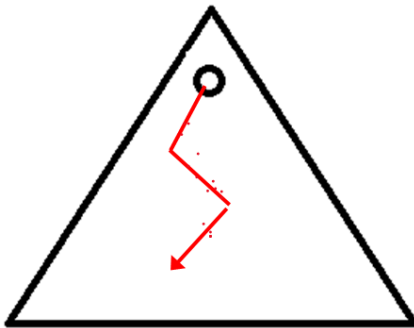
$\text{sink}(A, 0, 8)$

Complexity of the „buildMaxHeap” Algorithm

- It is easy to prove that building the heap is $O(n \log n)$.
- Although proving it is slightly more difficult, it can be shown that building the heap with the above method is: $\Theta(n)$.
- Why is it not a good approach to start from an empty heap and insert elements one by one using the rise operation?

Complexity of the „buildMaxHeap” Algorithm

- With that approach, building the heap would no longer be linear but $O(n \log n)$.
- Why?
- The bottom-up sinking method sinks fewer and fewer elements (always half as many) as the sinking path gets longer.
- The top-down rising method rises more and more elements (always twice as many) as the rising path gets longer.



A More Efficient Heap

- More efficient implementation of basic heap operations (rise, sink)
- The presented operations (rise and sink) use swaps. We know that every swap involves three moves. Algorithms can be made more efficient if they don't perform swaps, but rather "rise" or "sink" a "hole".
- For example, rising works by taking the element to be raised into a helper variable, then moving the "hole" upward in the heap by shifting the parent down if it is smaller. The hole thus moves to the parent's original position. This continues as long as necessary, and finally, the original element is placed into the hole.
- The sink operation can be optimized in the same way.

Emel(A/1:T[],k :N)

$x := A[k]$
$j := k \quad i := \lfloor k/2 \rfloor$
$i > 0 \wedge A[i] < x$
$A[j] := A[i]$
$j := i \quad i := \lfloor i/2 \rfloor$
$A[j] := x$

Optional Assignments

- Create a recursive algorithm that determines if a linked binary tree is a binary search tree. The tree consists of Node type elements; keys are natural numbers.
- Given an array $A[1..n]$ containing keys of arbitrary type, where the array is strictly monotonically increasing. Create a recursive algorithm that builds an optimally shaped binary search tree from the array. The tree should be represented with three pointers (using elements of type Node3*). The algorithm should create the necessary nodes on the fly and return a pointer to the root of the tree.
- Similarly, implement the Sink($A[1:T];k,n:N$) algorithm without direct swaps: copy the element to be sinked into a helper variable x , then sink the resulting "hole" into the heap. Once the final position is found, copy x into the "hole".

Thank you for your attention!